ARTI308 - Machine Learning

Linear Regression

Your neighbor is a real estate agent who wants to predict housing prices in different regions of the USA. Using the USA_Housing.csv dataset, you will build a Linear Regression model that predicts a house's selling price based on its features. Your neighbor then gives you some information about a bunch of houses in regions of the United States, it is all in the data set: USA_Housing.csv.

The data contains the following columns:

- 'Avg. Area Income': Avg. Income of residents of the city house is located in.
- 'Avg. Area House Age': Avg Age of Houses in same city
- 'Avg. Area Number of Rooms': Avg Number of Rooms for Houses in same city
- 'Avg. Area Number of Bedrooms': Avg Number of Bedrooms for Houses in same city
- 'Area Population': Population of city house is located in
- 'Price': Price that the house sold at
- 'Address': Address for the house

Let's get started!

Check out the data

We've been able to get some data from your neighbor for housing prices as a csv set, let's get our environment ready with the libraries we'll need and then import the data!

Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline
```

Check out the Data

```
USAhousing = pd.read_csv('USA_Housing.csv')
```

```
USAhousing.head()
```

	Avg. Area Income	Avg. Area House Age	Avg. Area Number of Rooms	Avg. Area Number of Bedrooms	Area Population	Price	Address
0	79545.458574	5.682861	7.009188	4.09	23086.800503	1.059034e+06	208 Michael Ferry Apt. 674\nLaurabury, NE 3701...
1	79248.642455	6.002900	6.730821	3.09	40173.072174	1.505891e+06	188 Johnson Views Suite 079\nLake Kathleen, CA...
2	61287.067179	5.865890	8.512727	5.13	36882.159400	1.058988e+06	9127 Elizabeth Stravenue\nDanieltown, WI 06482...
3	63345.240046	7.188236	5.586729	3.26	34310.242831	1.260617e+06	USS Barnett\nFPO AP 44820
4	59982.197226	5.040555	7.839388	4.23	26354.109472	6.309435e+05	USNS Raymond\nFPO AE

Next steps: Generate code with USAhousing | New interactive sheet

```
USAhousing.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5000 entries, 0 to 4999
Data columns (total 7 columns):
 #   Column                       Non-Null Count  Dtype  
---  ------                       --------------  -----  
 0   Avg. Area Income             5000 non-null   float64
 1   Avg. Area House Age          5000 non-null   float64
 2   Avg. Area Number of Rooms    5000 non-null   float64
 3   Avg. Area Number of Bedrooms 5000 non-null   float64
 4   Area Population              5000 non-null   float64
 5   Price                        5000 non-null   float64
 6   Address                      5000 non-null   object 
dtypes: float64(6), object(1)
memory usage: 273.6+ KB
```

```
USAhousing.describe()
```

	Avg. Area Income	Avg. Area House Age	Avg. Area Number of Rooms	Avg. Area Number of Bedrooms	Area Population	Price
count	5000.000000	5000.000000	5000.000000	5000.000000	5000.000000	5.000000e+03
mean	68583.108984	5.977222	6.987792	3.981330	36163.516039	1.232073e+06
std	10657.991214	0.991456	1.005833	1.234137	9925.650114	3.531176e+05
min	17796.631190	2.644304	3.236194	2.000000	172.610686	1.593866e+04
25%	61480.562388	5.322283	6.299250	3.140000	29403.928702	9.975771e+05
50%	68804.286404	5.970429	7.002902	4.050000	36199.406689	1.232669e+06
75%	75783.338666	6.650808	7.665871	4.490000	42861.290769	1.471210e+06
max	107701.748378	9.519088	10.759588	6.500000	69621.713378	2.469066e+06

```
USAhousing.columns
```

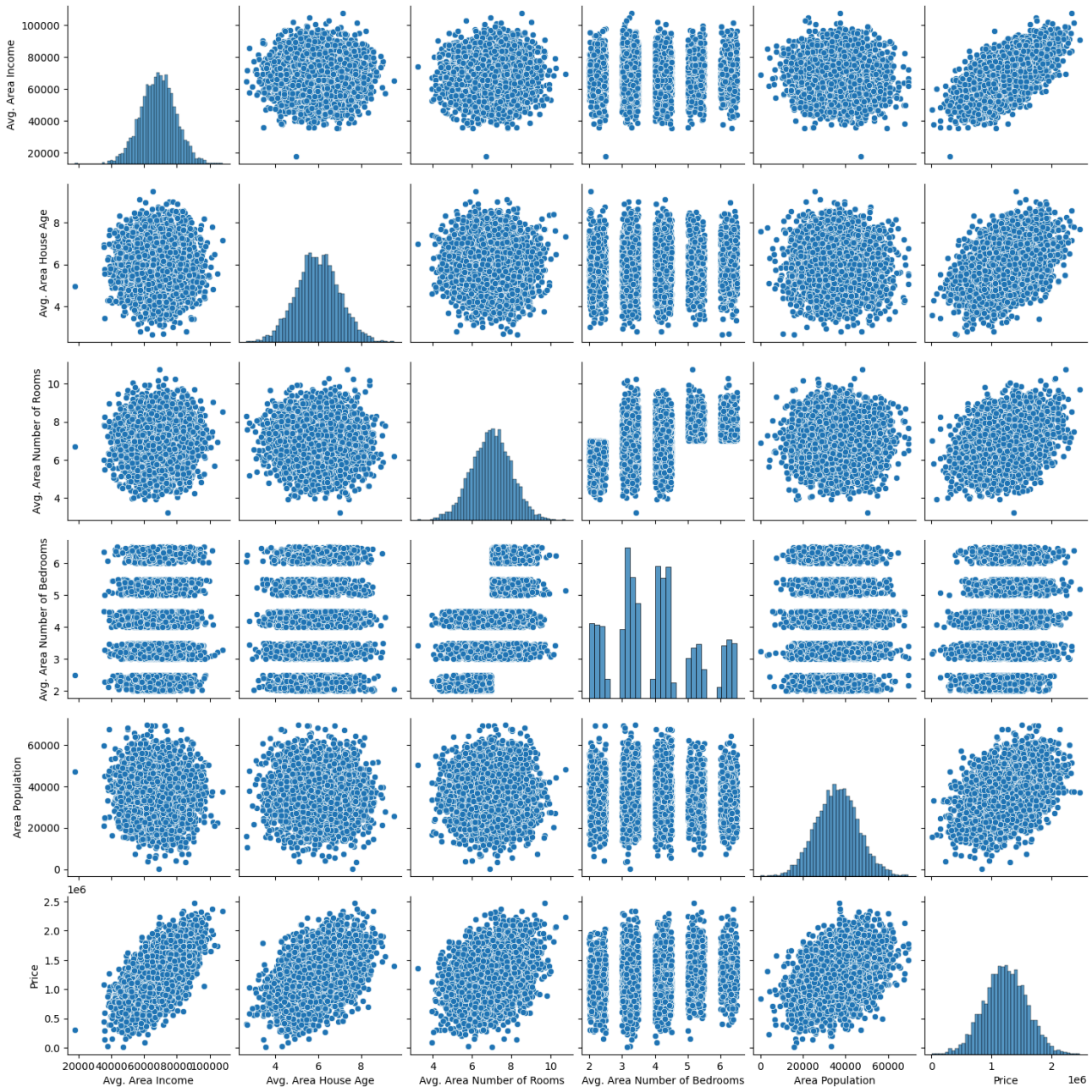
```
Index(['Avg. Area Income', 'Avg. Area House Age', 'Avg. Area Number of Rooms',
       'Avg. Area Number of Bedrooms', 'Area Population', 'Price', 'Address'],
      dtype='object')
```

EDA

Let's create some simple plots to check out the data!

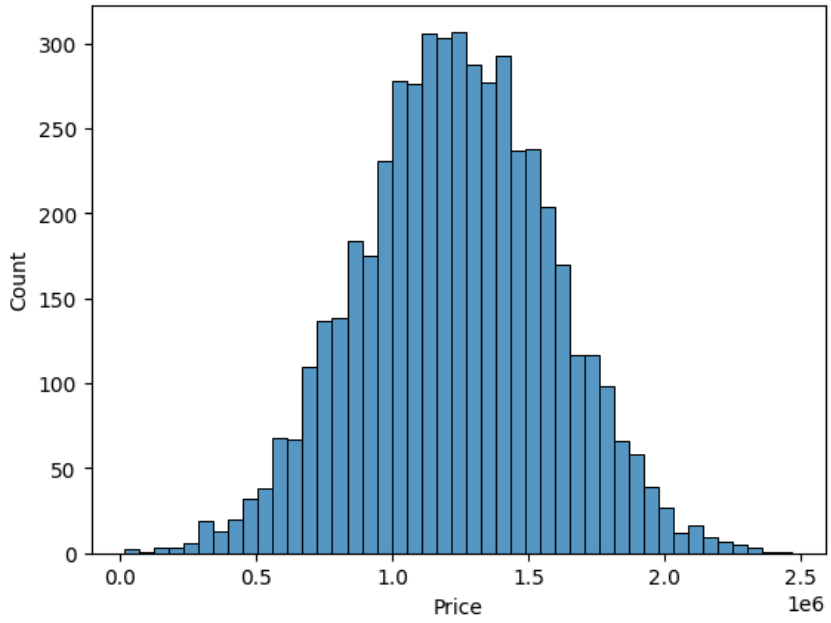
```
sns.pairplot(USAhousing)
```

```
<seaborn.axisgrid.PairGrid at 0x7c7a40f30590>
```



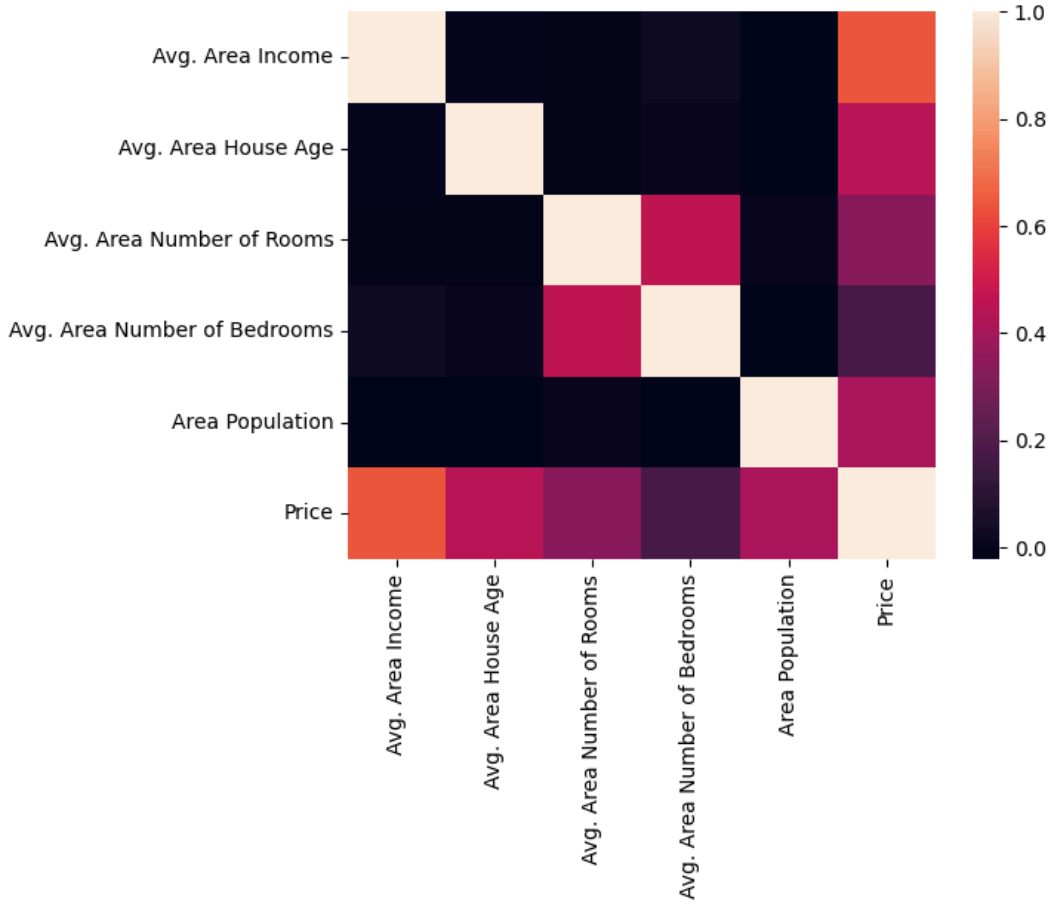

```
#sns.distplot(USAhousing['Price'])
sns.histplot(USAhousing['Price'])
```

```
<Axes: xlabel='Price', ylabel='Count'>
```




```
#sns.heatmap(USAhousing.corr()) this function produced
sns.heatmap(USAhousing.select_dtypes(include='number').corr())
```

```
<Axes: >
```



Training a Linear Regression Model

Let's now begin to train out regression model! We will need to first split up our data into an X array that contains the features to train on, and a y array with the target variable, in this case the Price column. We will toss out the Address column because it only has text info that the linear regression model can't use.

X and y arrays

```
X = USAhousing[['Avg. Area Income', 'Avg. Area House Age', 'Avg. Area Number of Rooms',
                'Avg. Area Number of Bedrooms', 'Area Population']]
y = USAhousing['Price']
```

Train Test Split

Now let's split the data into a training set and a testing set. We will train out model on the training set and then use the test set to evaluate the model.

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.4, random_state=101)
```

Creating and Training the Model

```
from sklearn.linear_model import LinearRegression
```

```
lm = LinearRegression()
```

```
lm.fit(X_train,y_train)
```

```
▾ LinearRegression
LinearRegression()
```

Model Evaluation

Let's evaluate the model by checking out its coefficients and how we can interpret them.

Important LinearRegression Attributes and Functions

Attribute / Function	Purpose	Example
`fit()`	Train the model	`lm.fit(X_train, y_train)`
`predict()`	Predict values for new data	`lm.predict(X_test)`
`coef_`	Shows learned coefficients (β_1, β_2, ...)	`lm.coef_`
`intercept_`	Shows intercept (β_0)	`lm.intercept_`
`score()`	Returns model accuracy (R^2 score)	`lm.score(X_test, y_test)`

```
# print the intercept
print(lm.intercept_)
```

```
-2640159.7968526953
```

```
coeff_df = pd.DataFrame(lm.coef_,X.columns,columns=['Coefficient'])
coeff_df
```

	Coefficient
Avg. Area Income	21.528276
Avg. Area House Age	164883.282027
Avg. Area Number of Rooms	122368.678027
Avg. Area Number of Bedrooms	2233.801864
Area Population	15.150420

Next steps: Generate code with `coeff_df` New interactive sheet

Interpreting the coefficients:

- Holding all other features fixed, a 1 unit increase in **Avg. Area Income** is associated with an **increase of $21.52 **.
- Holding all other features fixed, a 1 unit increase in **Avg. Area House Age** is associated with an **increase of $164883.28 **.
- Holding all other features fixed, a 1 unit increase in **Avg. Area Number of Rooms** is associated with an **increase of $122368.67 **.
- Holding all other features fixed, a 1 unit increase in **Avg. Area Number of Bedrooms** is associated with an **increase of $2233.80 **.
- Holding all other features fixed, a 1 unit increase in **Area Population** is associated with an **increase of $15.15 **.

Does this make sense? Probably not because I made up this data. If you want real data to repeat this sort of analysis, check out the boston dataset:

```
from sklearn.datasets import load_boston
boston = load_boston()
print(boston.DESCR)
boston_df = boston.data
```

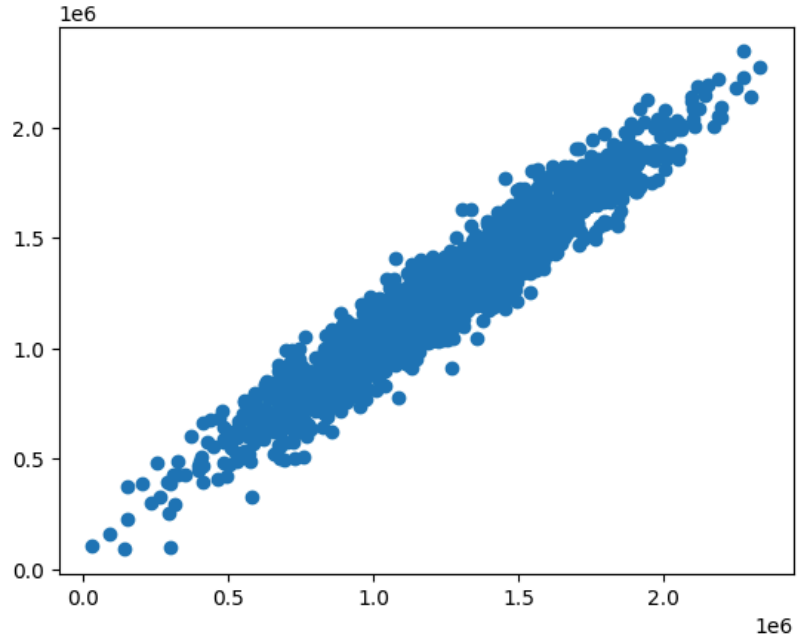
Predictions from our Model

Let's grab predictions off our test set and see how well it did!

```
predictions = lm.predict(X_test)
```

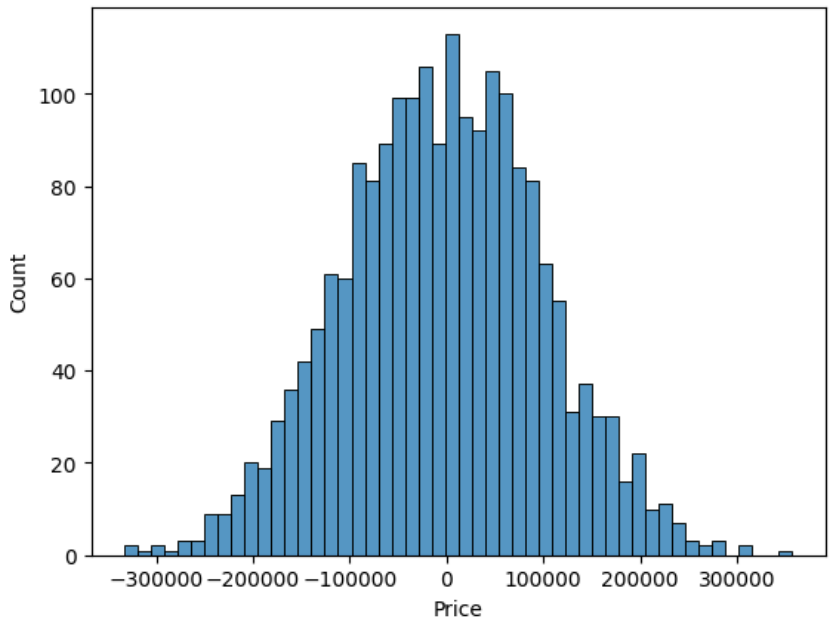
```
plt.scatter(y_test,predictions)
```

```
<matplotlib.collections.PathCollection at 0x7c7a35b86e90>
```



Residual Histogram

```
sns.histplot((y_test-predictions),bins=50);
```



Regression Evaluation Metrics

Here are three common evaluation metrics for regression problems:

Mean Absolute Error (MAE) is the mean of the absolute value of the errors:

$$\frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

Mean Squared Error (MSE) is the mean of the squared errors:

$$\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

Root Mean Squared Error (RMSE) is the square root of the mean of the squared errors:

$$\sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

Comparing these metrics:

- **MAE** is the easiest to understand, because it's the average error.
- **MSE** is more popular than MAE, because MSE "punishes" larger errors, which tends to be useful in the real world.
- **RMSE** is even more popular than MSE, because RMSE is interpretable in the "y" units.

All of these are **loss functions**, because we want to minimize them.

```
from sklearn import metrics
```

```
print('MAE:', metrics.mean_absolute_error(y_test, predictions))
print('MSE:', metrics.mean_squared_error(y_test, predictions))
print('RMSE:', np.sqrt(metrics.mean_squared_error(y_test, predictions)))
```

```
MAE: 82288.22251914942
MSE: 10460958907.208977
RMSE: 102278.82922290897
```

This was your first real Machine Learning Project! Congrats on helping your neighbor out!

Task:

In this task, you will apply what you have learned in this lab on a new dataset.

We will work with the Ecommerce Customers dataset (you will find it on Blackboard)

1. Load the dataset into a DataFrame
2. Explore the data (head, info, describe)
3. Perform basic data cleaning if needed
4. Apply feature engineering (if applicable)
5. Prepare the data for modeling
6. Train a model (use the same model used in the lab)
7. Evaluate the model performance

Task Solution: Ecommerce Customers

1. Load the dataset

```
customers = pd.read_csv('Ecommerce Customers.csv')
```

2. Explore the data

```
customers.head()
```

	Email	Address	Avatar	Avg. Session Length	Time on App	Time on Website	Length of Membership	
0	mstephenson@fernandez.com	835 Frank Tunnel\nWrightmouth, MI 82180-9605	Violet	34.497268	12.655651	39.577668	4.082621	58
1	hduke@hotmail.com	4547 Archer Common\nDiazchester, CA 06566-8576	DarkGreen	31.926272	11.109461	37.268959	2.664034	39
2	pallen@yahoo.com	24645 Valerie Unions Suite 582\nCobbborough, D...	Bisque	33.000915	11.330278	37.110597	4.104543	48
3	riverarebecca@gmail.com	1414 David Throughway\nPort Jason, OH 22070-1220	SaddleBrown	34.305557	13.717514	36.721283	3.120179	58
4	mstephens@davidson-herman.com	14023 Rodriguez Passage\nPort Jacobville, PR 3...	MediumAquaMarine	33.330673	12.795189	37.536653	4.446308	59

Next steps: Generate code with `customers` New interactive sheet

```
customers.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 8 columns):
 #   Column                Non-Null Count  Dtype  
---  ------                --------------  -----  
 0   Email                 500 non-null    object 
 1   Address               500 non-null    object 
 2   Avatar                500 non-null    object 
 3   Avg. Session Length   500 non-null    float64
 4   Time on App           500 non-null    float64
 5   Time on Website       500 non-null    float64
 6   Length of Membership  500 non-null    float64
 7   Yearly Amount Spent   500 non-null    float64
dtypes: float64(5), object(3)
memory usage: 31.4+ KB
```

```
customers.describe()
```

	Avg. Session Length	Time on App	Time on Website	Length of Membership	Yearly Amount Spent
count	500.000000	500.000000	500.000000	500.000000	500.000000
mean	33.053194	12.052488	37.060445	3.533462	499.314038
std	0.992563	0.994216	1.010489	0.999278	79.314782
min	29.532429	8.508152	33.913847	0.269901	256.670582
25%	32.341822	11.388153	36.349257	2.930450	445.038277
50%	33.082008	11.983231	37.069367	3.533975	498.887875
75%	33.711985	12.753850	37.716432	4.126502	549.313828
max	36.139662	15.126994	40.005182	6.922689	765.518462

```
customers.columns
```

```
Index(['Email', 'Address', 'Avatar', 'Avg. Session Length', 'Time on App',
       'Time on Website', 'Length of Membership', 'Yearly Amount Spent'],
      dtype='object')
```

3. Basic data cleaning

```
customers.isnull().sum()
```

	0
Email	0
Address	0
Avatar	0
Avg. Session Length	0
Time on App	0
Time on Website	0
Length of Membership	0
Yearly Amount Spent	0

dtype: int64

```
customers.duplicated().sum()
```

```
np.int64(0)
```

The dataset has no missing values and no duplicated rows, so no cleaning is needed.

4. Feature engineering

The columns `Email`, `Address` and `Avatar` are text and cannot be used by Linear Regression, so we drop them. The remaining features are already numeric. We also check the correlation with the target `Yearly Amount Spent`.

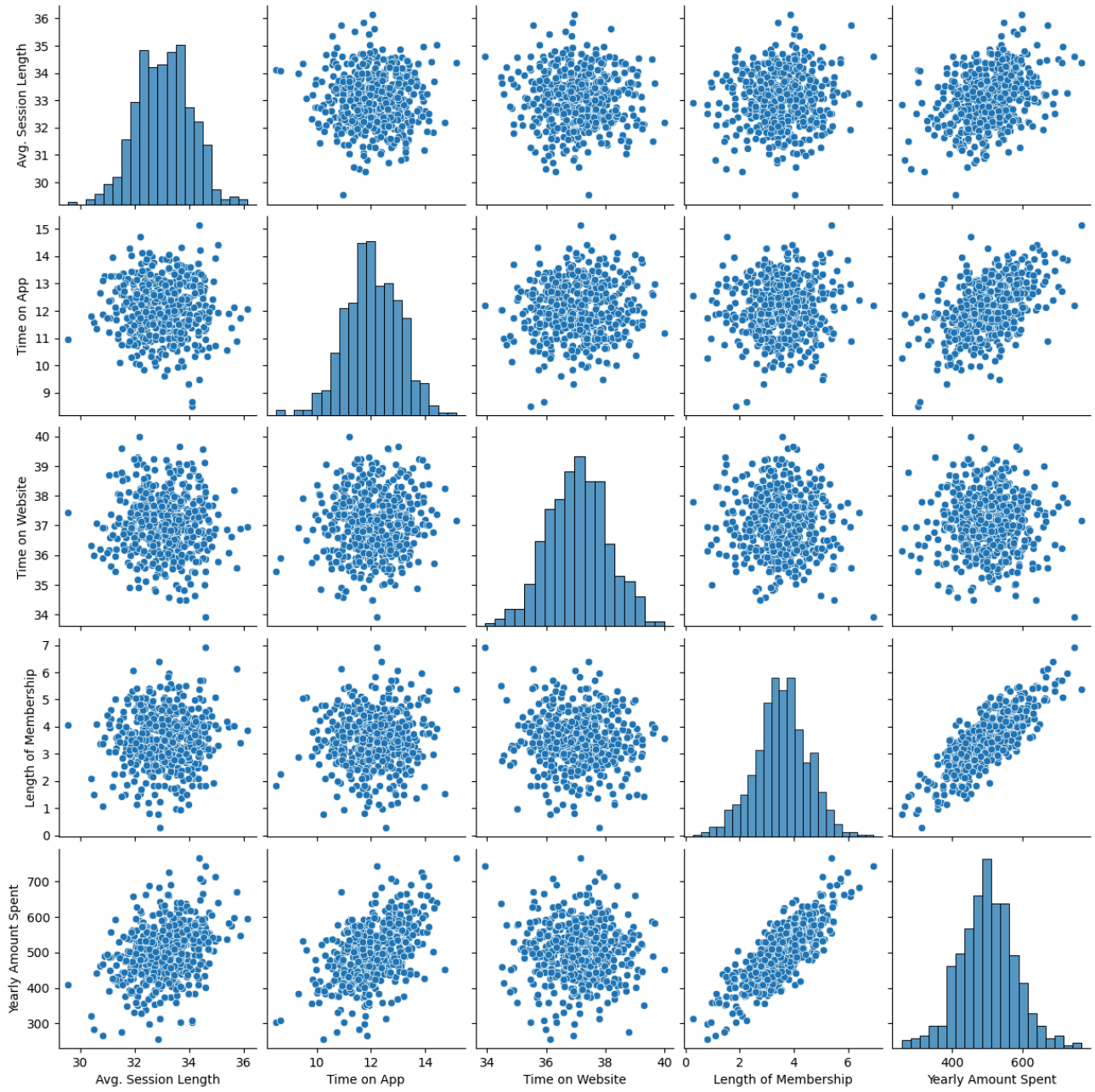
```
customers_num = customers.drop(['Email', 'Address', 'Avatar'], axis=1)
customers_num.head()
```

	Avg. Session Length	Time on App	Time on Website	Length of Membership	Yearly Amount Spent
0	34.497268	12.655651	39.577668	4.082621	587.951054
1	31.926272	11.109461	37.268959	2.664034	392.204933
2	33.000915	11.330278	37.110597	4.104543	487.547505
3	34.305557	13.717514	36.721283	3.120179	581.852344
4	33.330673	12.795189	37.536653	4.446308	599.406092

Next steps: Generate code with `customers_num` New interactive sheet

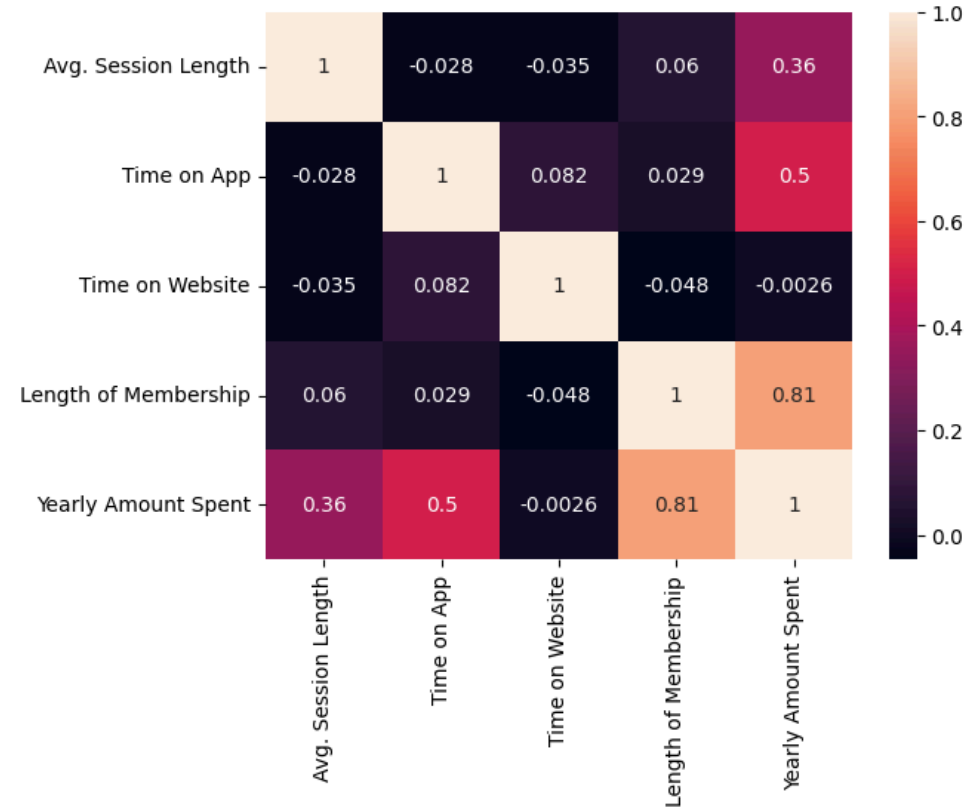
```
sns.pairplot(customers_num)
```

```
<seaborn.axisgrid.PairGrid at 0x7c7a35bee710>
```




```
sns.heatmap(customers_num.corr(), annot=True)
```

```
<Axes: >
```




```
customers_num.corr()['Yearly Amount Spent'].sort_values(ascending=False)
```

	Yearly Amount Spent
Yearly Amount Spent	1.000000
Length of Membership	0.809084
Time on App	0.499328
Avg. Session Length	0.355088
Time on Website	-0.002641

dtype: float64

`Length of Membership` has the strongest correlation with the target (0.81), followed by `Time on App` (0.50). `Time on Website` has almost no correlation (close to 0).

5. Prepare the data for modeling

```
X = customers[['Avg. Session Length', 'Time on App', 'Time on Website', 'Length of Membership']]
y = customers['Yearly Amount Spent']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.4, random_state=101)
```

```
print('Train size:', X_train.shape)
print('Test size:', X_test.shape)
```

```
Train size: (300, 4)
Test size: (200, 4)
```

6. Train the model

```
lm = LinearRegression()
```

```
lm.fit(X_train, y_train)
```

```
LinearRegression
LinearRegression()
```

```
print(lm.intercept_)
```

```
-1045.1152168245746
```

```
coeff_df = pd.DataFrame(lm.coef_, X.columns, columns=['Coefficient'])
coeff_df
```

	Coefficient
Avg. Session Length	25.691540
Time on App	37.892600
Time on Website	0.560581
Length of Membership	61.648594

Next steps: Generate code with coeff_df | New interactive sheet

Interpreting the coefficients:

- Holding all other features fixed, a 1 unit increase in **Avg. Session Length** is associated with an increase of about $25.69 * *inyearlyspending. - Holdingallotherfeaturesfixed, a1unitincreasein * *TimeonApp *$**37.89**$.*isassociatedwithanincreaseofabout * *$
- Holding all other features fixed, a 1 unit increase in **Time on Website** is associated with an increase of about $0.56 * *. - Holdingallotherfeaturesfixed, a1unitincreasein * *LengthofMembership *$**61.65**$.*isassociatedwithanincreaseofabout * *$

This means the mobile app and the length of membership have the biggest effect on spending, while the website has almost no effect.

7. Evaluate the model performance

```
predictions = lm.predict(X_test)
```